

Universidad Autónoma de Madrid
Escuela Politécnica Superior
Grado en Ingeniería Informática
Programación II
Hoja de ejercicios – Árboles Binarios

En los ejercicios que corresponda, asumir que el TAD ArbolBinario se ha implementado con la siguiente EdD:

```
// En arbolbinario.h

typedef struct _ArbolBinario ArbolBinario;

// En arbolbinario.c

#define info(pnodo) ((pnodo)->info)
#define izq(pnodo) ((pnodo)->izq)
#define der(pnodo) ((pnodo)->der)

#define root(pab) ((pab)->root)

struct _NodoAB {
    Elemento *info;
    struct _NodoAB *izq;
    struct _NodoAB *der;
};

typedef struct _NodoAB NodoAB;

struct _ArbolBinario {
    NodoAB *root;    // un árbol es el puntero a su nodo raíz
};
```

Ejercicio 1. Construye un árbol binario de búsqueda a partir de la siguiente lista:

G B Q A C K F P D E R H

Ejercicio 2. Para el árbol obtenido en el ejercicio 1 muestra el estado del mismo tras extraer de forma iterativa los siguientes elementos:

- a) E
- b) C
- c) G

Ejercicio 3. Para el árbol obtenido en el ejercicio 1 muestra la evolución de los siguientes algoritmos de recorrido de árboles binarios:

- d) Recorrido en preorden
- e) Recorrido en inorden
- f) Recorrido en postorden
- g) Recorrido en anchura

Ejercicio 4. Escribe el pseudocódigo de los algoritmos recursivos apropiados para determinar:

- a) La cantidad de nodos de un árbol binario
- b) El número de hojas de un árbol binario
- c) La profundidad de un árbol binario

Ejercicio 5. Escribe el código C de los algoritmos del ejercicio 3.

- a) La cantidad de nodos de un árbol binario: `int ab_numNodos(ArbolBinario *pab)`
- b) El número de hojas de un árbol binario: `int ab_numHojas(ArbolBinario *pab)`
- c) La profundidad de un árbol binario: `int ab_profundidad(ArbolBinario *pab)`

Ejercicio 6. Escribe el código C de la siguiente función que busca un elemento en un árbol binario y devuelve el nivel en el que se encuentra y -1 en caso de que no esté en el árbol.

```
int ab_buscar(ArbolBinario *pab, Elemento *pe)
```

Asumir que existe una función de comparación para el TAD Elemento

```
int elemento_comparar(Elemento *pe1, Elemento *pe2)
```

Ejercicio 7. Escribe el código C de siguiente función del ejercicio 5 para árboles binarios de búsqueda.

```
int abdb_buscar(ArbolBinario *pab, Elemento *pe)
```

Ejercicio 8. Dibuja los árboles de expresión que representan las siguientes expresiones:

- a) $A + B + C / D$
- b) $A - (B - (C - D) / (E + F)) ;$
- c) $(A + B) * ((C + D) / (E + F))$

Ejercicio 9. Construye algorítmicamente el árbol de expresión a partir de las siguientes expresiones postfijo:

- a) $A \ B \ C \ * \ D \ - \ E \ F \ / \ G \ / \ - \ *$
- b) $A \ B \ + \ C \ D \ - \ / \ E \ F \ * \ G \ - \ -$
- c) $A \ B \ C \ D \ - \ - \ E \ F \ G \ + \ + \ * \ *$

Ejercicio 10. Utilizando los árboles obtenidos en ejercicio 8, obtén la expresión prefijo de las expresiones asociadas.

Ejercicio 11. Utilizando los árboles obtenidos en ejercicio 8, obtén la expresión infijo de las expresiones asociadas.

Ejercicio 12. Dibuja la evolución del algoritmo HeapSort para la siguiente lista de elementos:

G B Q A C K F P D E R H

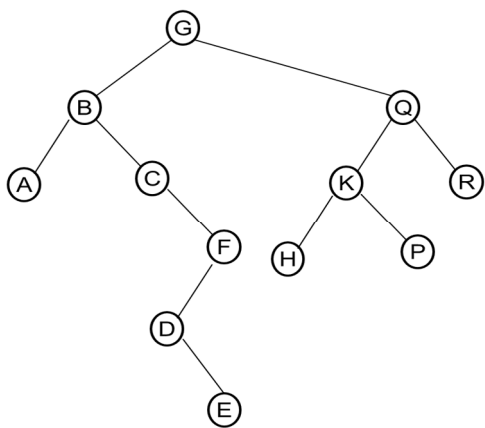
Considera por separado las dos versiones de creación del heap (`buildHeap`) vistas en clase:

- a) Inserción en el heap elemento a elemento de la lista.
- b) Creación in-place.

Considera también que se aplica el orden lexicográfico para la comparación de elementos, y que el heap construido es un MinHeap.

Soluciones

Ejercicio 1



Ejercicio 3

Preorden: G B A C F D E Q K H P R

Inorden: A B C D E F G H K P Q R

Postorden: A E D F C B H P K R Q G

Anchura: G B Q A C K R F H P D E

Ejercicio 4

<pre>// Numero de nodos int abNumNodos (ab T) if abVacio(T)==TRUE return 0 return 1 + abNumNodos(IZQ(T)) + abNumNodos(DER(T))</pre>	<pre>// Numero de hojas int numHojas (ab T) if abVacio(T) ==TRUE return 0 if abEsHoja(T) == TRUE return 1 return abNumHojas(IZQ(T)) + abNumHojas(DER(T)) BOOL esHoja (ab T) if (abVacio(IZQ(T))==TRUE AND abVacio(DER(T))==TRUE) return TRUE return FALSE</pre>
<pre>// Profundidad int abProf (ab T) if abVacio(T)==TRUE return -1 i = abProf(IZQ(t)) d = abProf(DER(T)) return 1 + maximo(i,d) int maximo (int a, int b) if (a>=b) return a else return b</pre>	

Ejercicio 5

```

int ab_numNodos(ArbolBinario *pab) {
    if (!pab)
        return 0;
    return ab_numNodos_rec(root(pab));
}

int ab_numNodos_rec(NodoAB *pn) {
    if (!pn)
        return 0;
    return 1 + ab_numNodos_rec(izq(pn)) + ab_numNodos_rec(der(pn));
}

-----

int ab_numHojas(ArbolBinario *pab) {
    if (!pab)
        return -1;
    return ab_numHojas_rec(root(pab));
}

int ab_numHojas_rec(NodoAB *pn) {
    int numHojasIzq, numHojasDer;

    if (!pn)
        return 0;

    if (!izq(pn) && !der(pn))
        return 1;

    numHojasIzq = ab_numHojas_rec(izq(pn));
    numHojasDer = ab_numHojas_rec(der(pn));

    return numHojasIzq + numHojasDer;
}

-----

int ab_prof(ArbolBinario *pab) {
    if (!pab)
        return 0;
    return ab_prof_rec(root(pab));
}

int ab_prof_rec(NodoAB *pn) {
    int profIzq, profDer;

    if (!pn)
        return -1;

    profIzq = ab_prof_rec(izq(pn));
    profDer = ab_prof_rec(der(pn));

    return 1 + max(profIzq, profDer);
}

int max(int a, int b) {
    return a >= b ? a : b;
}

```

Ejercicio 6

```

int ab_buscar(ArbolBinario *pab, Elemento *pe) {
    if (!pab || !pe)
        return -1;

    return ab_buscar_rec(root(pab), pe);
}

int ab_buscar_rec(NodoAB *pn, Elemento *pe) {
    int nivel;

    if (!pab || !pe)
        return -1;

    if (elemento_comparar(info(pn), pe) == 0)
        return 0;

    nivel = ab_buscar_rec(izq(pn), pe); // Se ha encontrado por el subárbol izquierdo
    if (nivel != -1)
        return 1 + nivel;

    nivel = ab_buscar_rec(der(pn), pe); // Se ha encontrado por el subárbol derecho
    if (nivel != -1)
        return 1 + nivel;

    return -1;
}

```

Ejercicio 7

```

int abdb_buscar(ArbolBinario *pab, Elemento *pe) {
    if (!pab || !pe)
        return -1;

    return abdb_buscar_rec(root(pab), pe);
}

int abdb_buscar_rec(NodoAB *pn, Elemento *pe) {
    int cmp, nivel;

    if (!pab || !pe)
        return -1;

    cmp = elemento_comparar(pe, info(pn));

    if (cmp == 0)
        return 0;

    if (cmp < 0)
        nivel = abdb_buscar_rec(izq(pn), pe);
    else
        nivel = abdb_buscar_rec(der(pn), pe);

    if (nivel != -1)
        return 1 + nivel;

    return -1;
}

```